Distances Between Probability Measures

Student Name: Anthony Matar

In this notebook, we will explore different ways to measure distances between probability distributions.

In [32]:
```
# If running on Google Colab, you might need to install POT (Python Optimal
# !pip install POT
```

In [33]:
```
pip install ot
```

```
Defaulting to user installation because normal site-packages is not writeabl
e
ERROR: Could not find a version that satisfies the requirement ot (from vers
ions: none)
ERROR: No matching distribution found for ot
Note: you may need to restart the kernel to use updated packages.
```

In [34]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
import ot

# Plotting settings
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['axes.grid'] = True
```

Warm-Up: Wasserstein distance in 1D

In the lectures, you have seen that the 1D wasserstein distance has a closed-form solution that can be expressed via the quantile functions:

$$W_p(\mu, \nu) = \left(\int_0^1 |Q_\mu(t) - Q_\nu(t)|^p dt \right)^{1/p}$$

where Q_μ, Q_ν are the quantile (inverse CDF) functions.

Exercise 1. Implement the 1D wasserstein-p distance between 2 point clouds (of potentially different sizes n and m)

Hint: Because the empirical distributions are discrete, their quantile functions $Q(t)$ are step functions. If $n = m$, you could simply sort both arrays and compute the distance element-wise. However, when $n \neq m$, the "steps" of the quantile functions don't align.

To compute the exact integral $\int_0^1 |Q_\mu(t) - Q_\nu(t)|^p dt$, you need to evaluate the area of the piecewise constant segments.

Suggested Steps:

- Sort both arrays.
- Define the CDF levels. Create arrays representing the probability levels for each distribution: $1/n, 2/n, \ldots, 1$ and $1/m, 2/m, \ldots, 1$ (e.g. use `np.arange`).
- Merge the CDF levels by using `np.concatenate` + `np.unique`. These represent the boundaries on the $[0, 1]$ interval where *either* quantile function might jump.
- Evaluate both quantile functions at these levels (Tip: `np.searchsorted` is highly efficient for finding which quantile step a probability level falls into; e.g. `u_quantiles = u_sorted[np.searchsorted(u_cdf, all_levels, side='left')]`).
- Compute the widths of each probability interval (Tip: you can use `np.diff` on an array `np.concatenate([0.0], all_levels)`).
- Compute the integral as the sum of `width * |u_quantiles - v_quantiles|^p` (i.e. a weighted sum).
- Return the (p)-th root (unless `return_pth_power=True`).

In [35]:
```python
def wasserstein_p_1d(u: np.ndarray, v: np.ndarray, p: float = 2, return_pth_
    """
    Compute the 1D Wasserstein-p distance between two point clouds (of possi

    Args:
        u: 1D array of samples from the first distribution; of shape (n,).
        v: 1D array of samples from the second distribution; of shape (m,).
        p: Order of the distance (p >= 1). Defaults to 2.
        return_pth_power: If True, return the p-th power of the distance
            instead of the distance itself. Defaults to False.

    Returns:
        The Wasserstein-p distance as a float (or its pth power).
    """
    if p < 1:
        raise ValueError(f"p must be >= 1, got {p}")

    u = np.asarray(u, dtype=float).ravel()
    v = np.asarray(v, dtype=float).ravel()

    n, m = len(u), len(v)
    if n == 0 or m == 0:
        raise ValueError("Input arrays must be non-empty")

    # Sort both arrays
    u_sorted = np.sort(u)
    v_sorted = np.sort(v)

    # CDF levels: the i-th sample covers up to (i+1)/n probability mass
    u_cdf = np.arange(1, n + 1) / n
```

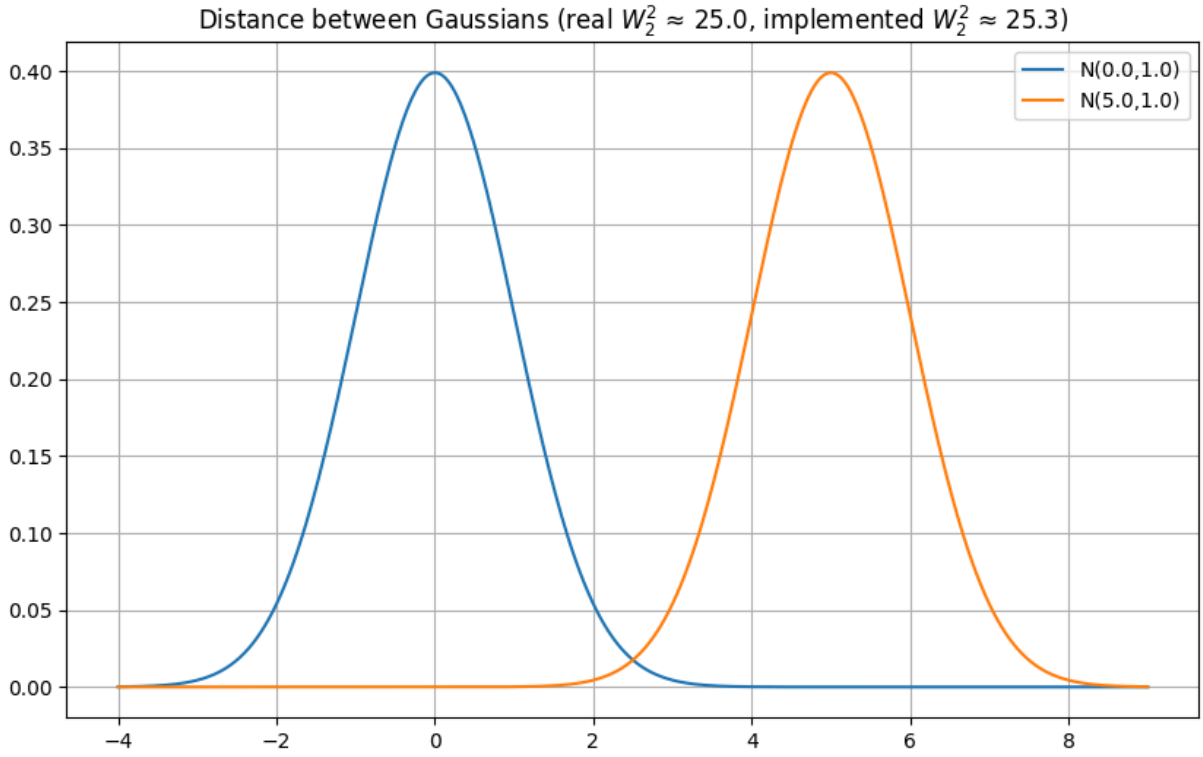
```
    v_cdf = np.arange(1, m + 1) / m

    # Merge all CDF breakpoints — these are where either quantile function j
    all_levels = np.unique(np.concatenate([u_cdf, v_cdf]))

    # Evaluate quantile functions: Q(t) = sorted[k] where k is the first inc
    # with cdf[k] >= t (i.e. the step that "covers" probability level t)
    u_quantiles = u_sorted[np.searchsorted(u_cdf, all_levels, side='left')]
    v_quantiles = v_sorted[np.searchsorted(v_cdf, all_levels, side='left')]

    # Width of each probability interval [prev_level, level]
    widths = np.diff(np.concatenate([[0.0], all_levels]))

    # Riemann sum: integral of |Q_u(t) - Q_v(t)|^p dt
    integral = float(np.sum(widths * np.abs(u_quantiles - v_quantiles) ** p)

    return integral if return_pth_power else float(integral ** (1.0 / p))
```

In 1D, for two Gaussians $\mu_1 = \mathcal{N}(\mu_1, \sigma_1^2)$ and $\mu_2 = \mathcal{N}(\mu_2, \sigma_2^2)$, we exactly have $W_2^2(\mu_1, \mu_2) = (\mu_1 - \mu_2)^2 + (\sigma_1 - \sigma_2)^2$.

Exercise 2. Generate samples from two 1D Gaussians $\mathcal{N}(\mu_1, \sigma_1^2)$ ($n = 10000$) and $\mathcal{N}(\mu_2, \sigma_2^2)$ ($m = 20000$) (you may choose μ_i, σ_i as you wish!) And use this to verify that your implementation of `wasserstein_p_1d` is correct by comparing its output with the formula provided above (bear in mind that your implementation is only an approximation of the real distance!).

In [36]:
```
def sanity_check_wasserstein_1d(mu1=0.0, sigma1=1.0, mu2=5.0, sigma2=1.0, n=
    rng = np.random.default_rng(42)
    u = rng.normal(mu1, sigma1, n)
    v = rng.normal(mu2, sigma2, m)

    # Exact closed-form W2^2 between two 1D Gaussians
    w2_analytic = (mu1 - mu2) ** 2 + (sigma1 - sigma2) ** 2

    # Empirical estimate via our implementation
    w2_empirical = wasserstein_p_1d(u, v, p=2, return_pth_power=True)

    # Plot densities
    t = np.linspace(min(mu1, mu2) - 4 * max(sigma1, sigma2),
                    max(mu1, mu2) + 4 * max(sigma1, sigma2), 500)
    plt.plot(t, norm.pdf(t, mu1, sigma1), label=f'N({mu1},{sigma1})')
    plt.plot(t, norm.pdf(t, mu2, sigma2), label=f'N({mu2},{sigma2})')
    plt.title(f"Distance between Gaussians (real $W_2^2$ ≈ {w2_analytic:.1f}
    plt.legend()
    plt.show()
```

In [37]:
```
n, m = 10000, 20000
# example values! You may change these as you like.
mu1, sigma1 = 0.0, 1.0
mu2, sigma2 = 5.0, 1.0

sanity_check_wasserstein_1d(mu1, sigma1, mu2, sigma2, n, m)
```



Distances between probability measures in higher dimensions

In [38]:
```python
## UTILITY FUNCTION
def _ensure_arrays(x,y):
    x = np.asarray(x, dtype=float)
    y = np.asarray(y, dtype=float)

    if x.ndim == 1:
        x = x[:, None]
    if y.ndim == 1:
        y = y[:, None]
    return x, y
```

Wasserstein Distance

The **Wasserstein distance** measures the optimal transport cost for moving mass from one distribution to the other (using dist^p). Formally, for two distributions μ and ν on $\mathbb{R}^d$, the p-Wasserstein distance is:

$$W_p(\mu,\nu) := \left(\inf_{\gamma \in \Pi(\mu,\nu)} \int \|x-y\|^p \, d\gamma(x,y) \right)^{1/p}$$

where $\Pi(\mu,\nu)$ is the set of couplings with marginals μ and ν.

Given empirical samples, this becomes a finite linear program, that we can solve using the dedicated python library `POT` (see here). The usual way of doing this is:

- Build the cost matrix $C_{ij} = \|x_i - y_j\|^p$ by using `C = ot.dist(x, y, metric='euclidean') ** p`.
- Solve the OT problem `float(ot.emd2(a, b, C))`, where `a` and `b` are arrays of size n and m respectively, which contain uniform weights for each sample (i.e. $a_i = 1/n$, $b_j = 1/m$).
- Take the p-th root (if `return_pth_power=False`).

Exercise 3. Complete the function below to implement the Wasserstein p distance between 2 point clouds.

In [39]:
```python
def wasserstein_p_empirical(x, y, p=2, return_pth_power=False):
    """Compute the empirical Wasserstein-p distance between two point clouds
    x: array of shape (n, d) - samples from the first distribution
    y: array of shape (m, d) - samples from the second distribution
    p: Order of the distance (p >= 1). Defaults to 2.
    return_pth_power: If True, return the p-th power of the distance
        instead of the distance itself. Defaults to False.
    """
    if p < 1:
        raise ValueError(f"p must be >= 1, got {p}")

    x, y = _ensure_arrays(x, y)
    n, m = x.shape[0], y.shape[0]

    a = np.ones(n) / n
    b = np.ones(m) / m
    C = ot.dist(x, y, metric='euclidean') ** p

    wp_p = float(ot.emd2(a, b, C))
    return wp_p if return_pth_power else wp_p ** (1.0 / p)
```

Sliced Wasserstein (SW) distance

The **Sliced Wasserstein (SW)** distance is an attempt at circumventing the curse of dimensionality of the Wasserstein distance (we will see this in the next section) by reducing the problem to a collection of cheap one-dimensional problems. For $\theta \in \mathbb{S}^{d-1}$, let $\Pi^\theta(x) = \langle \theta, x \rangle$ be the scalar projection of x onto direction θ. The pushed-forward measure $\Pi^\theta_\# \mu$ is simply the distribution of those projected scalars. The sliced Wasserstein distance then averages the 1D Wasserstein cost over all directions:

$$\mathrm{SW}_p(\mu, \nu) := \left(\int_{\mathbb{S}^{d-1}} W_p^p \left(\Pi^\theta_\# \mu,\, \Pi^\theta_\# \nu \right) \sigma(d\theta) \right)^{1/p}$$

where σ is the uniform measure on $\mathbb{S}^{d-1}$.

For implementing this, the integral over the sphere is approximated by averaging over L random directions, each drawn uniformly from $\mathbb{S}^{d-1}$. In 1D, W_p^p reduces to a simple sort-and-match, making each projection $O(n \log n)$ — far cheaper than solving the full OT problem in d dimensions.

Exercise 4. Complete the function below to implement the Sliced Wasserstein p distance between 2 point clouds. You may use the `wasserstein_p_1d` distance implemented in a previous exercise.

In [40]:
```python
def sliced_wasserstein_p(x, y, n_projections=50, p=2, return_pth_power=False
    """Compute the empirical Sliced-Wasserstein-p between two point clouds (
    x: array of shape (n, d) - samples from the first distribution
    y: array of shape (m, d) - samples from the second distribution
    p: Order of the distance (p >= 1). Defaults to 2.
    return_pth_power: If True, return the p-th power of the distance
        instead of the distance itself. Defaults to False.
    """
    if p < 1:
        raise ValueError(f"p must be >= 1, got {p}")

    x, y = _ensure_arrays(x, y)
    d = x.shape[1]

    # Sample random unit directions uniformly from S^{d-1}
    # by normalising Gaussian draws (the standard way)
    directions = np.random.randn(n_projections, d)
    directions /= np.linalg.norm(directions, axis=1, keepdims=True)

    # Average W_p^p over all projections
    swp_p = 0.0
    for theta in directions:
        x_proj = x @ theta   # shape (n,)
        y_proj = y @ theta   # shape (m,)
        swp_p += wasserstein_p_1d(x_proj, y_proj, p=p, return_pth_power=True
    swp_p /= n_projections

    return swp_p if return_pth_power else swp_p ** (1.0 / p)
```

The Maximum Mean Discrepancy (MMD)

The **Maximum Mean Discrepancy (MMD)** is a kernel-based quantity that measures the "closeness" between probability distributions (but it **doesn't necessarily define a distance!**). Given two distributions μ and ν on $\mathbb{R}^d$, the MMD is defined as the largest difference in expected values of a function f drawn from the unit ball of a reproducing kernel Hilbert space (RKHS) $\mathcal{H}$:

$$\mathrm{MMD}(\mu, \nu) = \sup_{\substack{f \in \mathcal{H} \\ \|f\|_{\mathcal{H}} \leq 1}} \left| \int f \, d\mu - \int f \, d\nu \right|$$

Expressing elements of the RKHS via the corresponding kernel, this admits the equivalent representation:

$$\mathrm{MMD}^2(\mu,\nu) = \iint k(x,y)\,\mu(dx)\mu(dy) + \iint k(x,y)\,\nu(dx)\nu(dy) - 2\iint k(x,y)\,\mu(dx)$$

Given i.i.d. samples $X_1,\dots,X_m \sim \mu$ and $Y_1,\dots,Y_n \sim \nu$, we can estimate this quantity via the empirical estimator, in which we simply replace integrals with sample averages over the kernel matrix (i.e. `kernel(x, x).mean()`, `kernel(y, y).mean()` and `kernel(x, y).mean()`).

Exercise 5. Complete the function below to implement the MMD between two point clouds. We have provided some kernel functions commonly used in practice.

In [41]:
```python
## IMPLEMENTING SOME KERNELS FOR MMD
def rbf(a, b, sigma=1.0):
    return np.exp(-ot.dist(a, b, metric='sqeuclidean') / (2 * sigma**2))

def laplacian(a, b, sigma=1.0):
    return np.exp(-ot.dist(a, b, metric='euclidean') / sigma)

def linear(a, b):
    return a @ b.T

def poly(a, b, degree=3, c=1.0):
    return (a @ b.T + c) ** degree
```

In [42]:
```python
def mmd(x, y, kernel=rbf, return_squared=False):
    """Compute the empirical MMD (according to a pluggable kernel) between t
    x: array of shape (n, d) - samples from the first distribution
    y: array of shape (m, d) - samples from the second distribution
    kernel: The kernel function to use. Defaults to the RBF kernel.
    return_squared: If True, return the square of the MMD instead of the MMD
    """
    x, y = _ensure_arrays(x,y)

    # MMD^2 = E_{x,x'}[k(x,x')] + E_{y,y'}[k(y,y')] - 2*E_{x,y}[k(x,y)]
    mmd2 = kernel(x, x).mean() + kernel(y, y).mean() - 2 * kernel(x, y).mear
    # Clip to 0 for numerical stability (mmd2 can be tiny-negative due to fl
    mmd2 = float(max(0.0, mmd2))
    return mmd2 if return_squared else float(np.sqrt(mmd2))
```

Just for completeness, we provide an approximate implementation of the Total Variation Distance.

In [43]:
```python
from scipy.stats import gaussian_kde

def tv_distance(x, y, n_points=10000):
    """TV distance approximation via KDE (to estimate the densities)."""
    x, y = _ensure_arrays(x,y)
    if x.shape[1] != y.shape[1]:
        raise ValueError("x and y must have the same ambient dimension")
```

```
    kde_x = gaussian_kde(x.T, 0.01)
    kde_y = gaussian_kde(y.T, 0.01)

    pooled = np.concatenate([x, y], axis=0)
    idx = np.random.choice(pooled.shape[0], size=min(n_points, len(pooled)),
    points = pooled[idx].T

    p = kde_x(points)
    q = kde_y(points)

    w = 0.5 * (p + q)
    return np.mean(np.abs(p - q) / w)
```

Putting it all together

Exercise 6. Complete the function `sample_clouds` below, which generates two point clouds in $\mathbb{R}^2$ with `n_samples` points:

1. x: a mixture of 3 Gaussians centered at the vertices of an equilateral triangle inscribed in a circle of radius `vertex_radius`, each with noise standard deviation `noise_scale` (this parameter won't be playing an important role). In other words, the means of these gaussians are placed at $(r\cos(\theta), r\sin(\theta))$, where r = `vertex_radius` and θ is one of $\{0, 2\pi/3, 4\pi/3\}$.
2. y: a single isotropic Gaussian centered at the origin with standard deviation `gaussian_std`.

As a sanity check, use the provided plotting function to visualize the obtained point clouds. For instance, check that when taking `vertex_radius = 0` and `gaussian_std = noise_scale`, the two point clouds *look indistinguishable*.

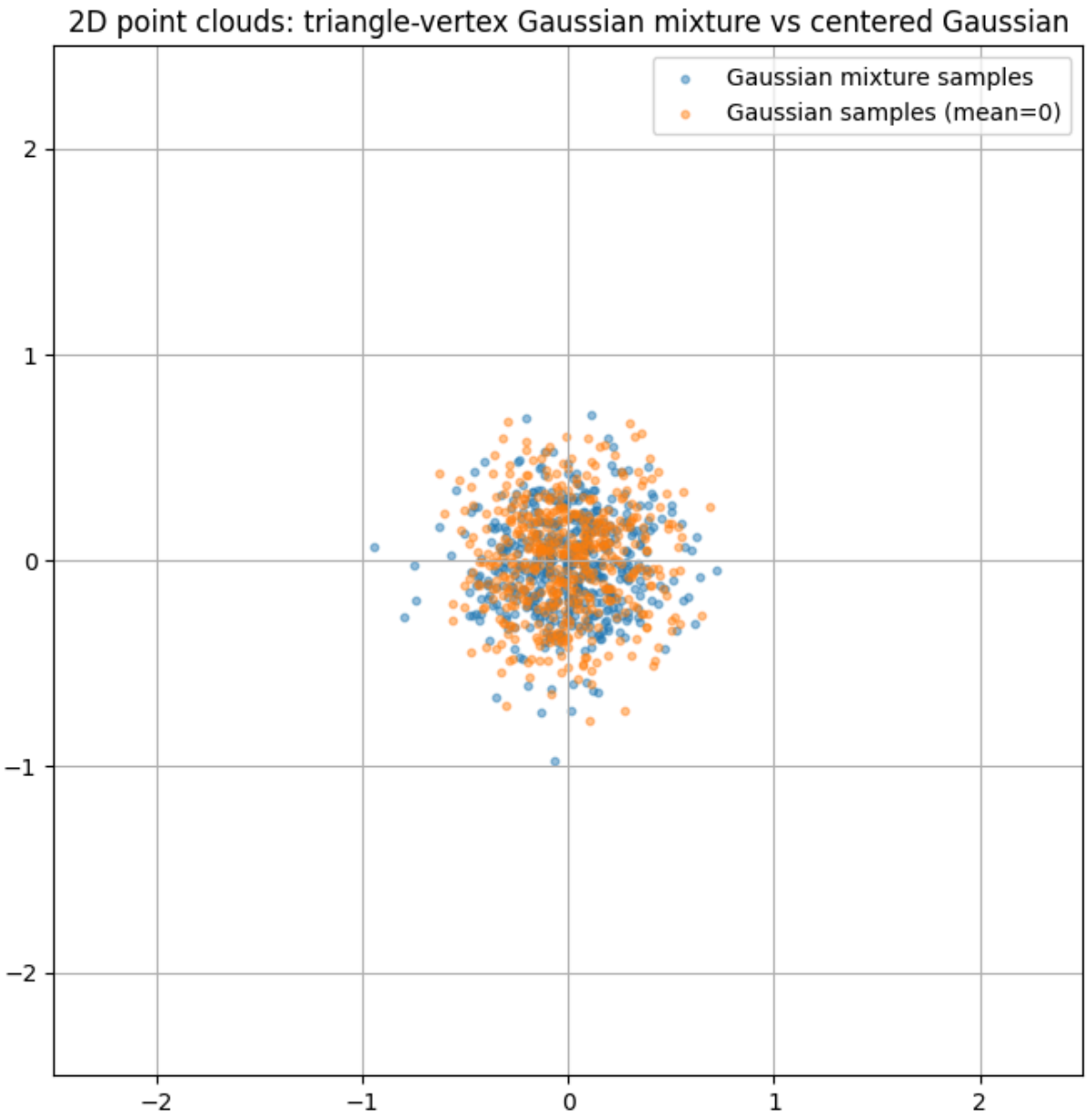
In [44]:
```
def sample_clouds(vertex_radius, gaussian_std, n_samples, noise_scale=0.25,
    """Generate a pair of point clouds: triangle-vertex Gaussian mixture and

    Args:
        vertex_radius: Distance from origin to each triangle vertex.
        gaussian_std: Standard deviation of the centered Gaussian.
        n_samples: Number of samples to generate for each distribution.
        noise_scale: Standard deviation of noise added to triangle vertices.
        seed: Random seed for reproducibility.

    Returns:
        x: Point cloud as 3-component Gaussian mixture (n_samples, 2).
        y: Point cloud as centered Gaussian (n_samples, 2).
    """
    rng = np.random.default_rng(seed)

    # Vertices of equilateral triangle inscribed in circle of radius vertex_
    angles = np.array([0.0, 2 * np.pi / 3, 4 * np.pi / 3])
    centers = vertex_radius * np.column_stack([np.cos(angles), np.sin(angles
```

```
    # Sample uniformly from the 3 components, then add Gaussian noise
    component_ids = rng.integers(0, 3, size=n_samples)
    x = centers[component_ids] + rng.normal(0.0, noise_scale, size=(n_sample

    # Centered isotropic Gaussian
    y = rng.normal(0.0, gaussian_std, size=(n_samples, 2))

    return x, y
```

In [45]:
```
def plot_2d_clouds(x2d, y2d):
    # Visualize both point clouds
    plt.figure(figsize=(8, 8))
    plt.scatter(x2d[:, 0], x2d[:, 1], s=10, alpha=0.45, label='Gaussian mixt
    plt.scatter(y2d[:, 0], y2d[:, 1], s=10, alpha=0.45, label='Gaussian samp
    plt.gca().set_aspect('equal', 'box')
    plt.title('2D point clouds: triangle-vertex Gaussian mixture vs centered
    plt.xlim(-2.5, 2.5)
    plt.ylim(-2.5, 2.5)
    plt.legend()
    plt.show()
```

In [46]:
```
# Generate point clouds
x2d, y2d = sample_clouds(
    vertex_radius=1.5,
    gaussian_std=0.75,
    n_samples=500
)

plot_2d_clouds(x2d, y2d)
```

In [47]:
```
# Generate point clouds
x2d, y2d = sample_clouds(
    vertex_radius=0,
    gaussian_std=0.25,
    n_samples=500
)

plot_2d_clouds(x2d, y2d)
```



Exercise 7. Complete the function `compute_metric_grids` below, which evaluates several distributional metrics over a 2D parameter grid. For each pair `gaussian_std_i`, `vertex_radius_j` in the grid, it generates two point clouds using the function `sample_clouds` and evaluates each distance metric in `metrics`, storing results in a dictionary of 2D arrays of shape `len(std_grid)`, `len(radius_grid)`.

In [48]:
```
def compute_metric_grids(metrics, std_grid, radius_grid, n_heatmap = 180):
    """ Compute the metric values over a grid of (gaussian_std, vertex_radiu
    metrics: dict of metric_name -> metric_function(x,y) that computes a dis
    std_grid: 1D array of gaussian_std values to evaluate.
    radius_grid: 1D array of vertex_radius values to evaluate.
    """

    grids = {name: np.full((len(std_grid), len(radius_grid)), np.nan, dtype=
    for i, gaussian_std_i in enumerate(std_grid):
```

```
        for j, vertex_radius_j in enumerate(radius_grid):
            x, y = sample_clouds(vertex_radius_j, gaussian_std_i, n_heatmap)
            for name, metric in metrics.items():
                try:
                    grids[name][i, j] = metric(x, y)
                except Exception:
                    grids[name][i, j] = np.nan
    return grids
```

Exercise 8. Run `compute_metric_grids` on a given metric_catalog containing metrics such as: W_1; W_2; SW_1; SW_2; MMD with RBF, Laplacian, Linear and Polynomial kernels; and the (approximate) Total Variation distance.

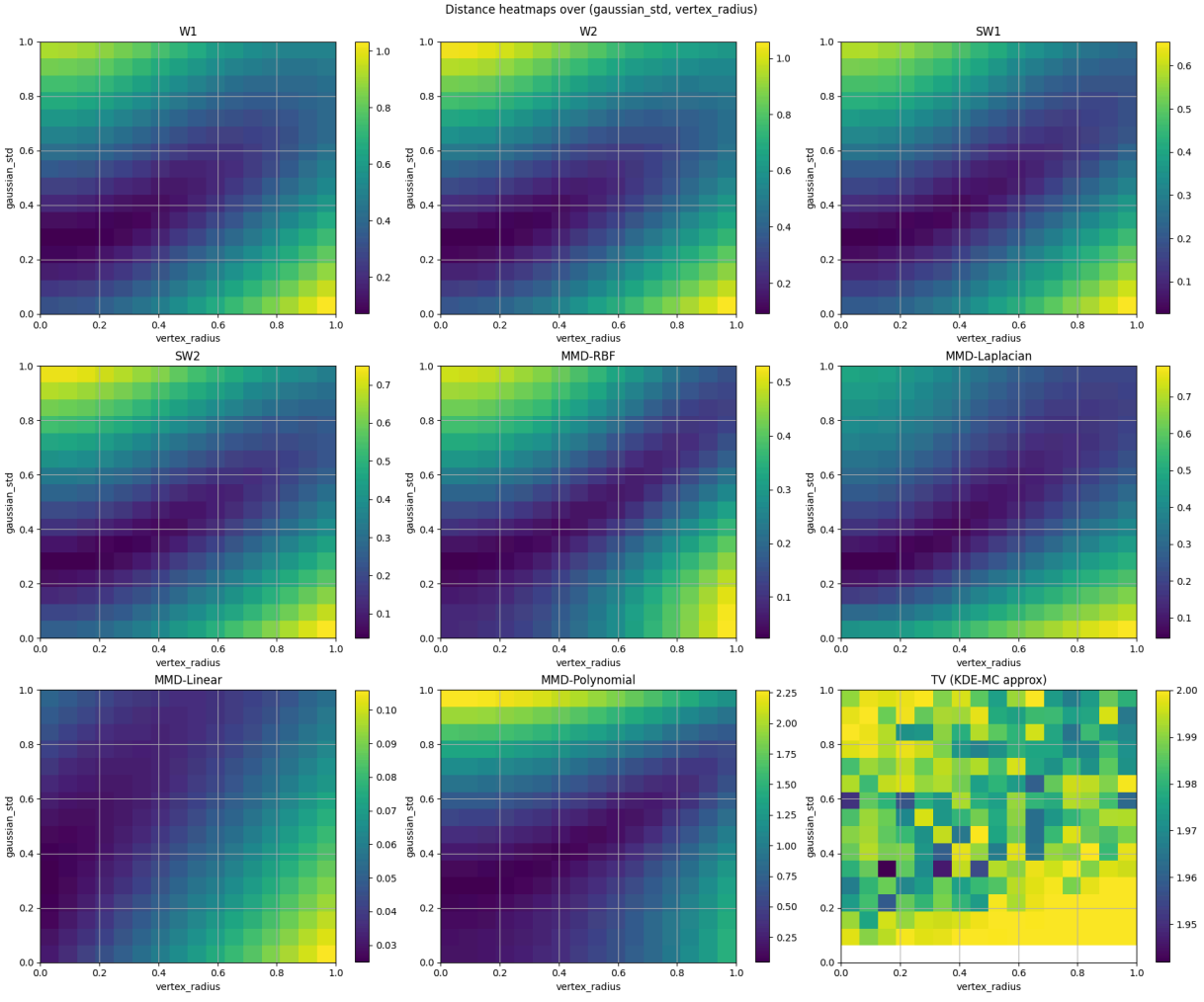
Use the plotting function to visualize the obtained grid. Comment on the obtained plot: Do all metrics behave similarly? What qualitative differences do you observe?

In [49]:
```
# Flexible heatmap panel over (gaussian_std, vertex_radius)
grid_size = 16
param_min, param_max = 0.0, 1.0
std_vals = np.linspace(param_min, param_max, grid_size)
radius_vals = np.linspace(param_min, param_max, grid_size)

metric_catalog = {
    "W1":              lambda x, y: wasserstein_p_empirical(x, y, p=1),
    "W2":              lambda x, y: wasserstein_p_empirical(x, y, p=2),
    "SW1":             lambda x, y: sliced_wasserstein_p(x, y, p=1),
    "SW2":             lambda x, y: sliced_wasserstein_p(x, y, p=2),
    "MMD-RBF":         lambda x, y: mmd(x, y, kernel=rbf),
    "MMD-Laplacian":   lambda x, y: mmd(x, y, kernel=laplacian),
    "MMD-Linear":      lambda x, y: mmd(x, y, kernel=linear),
    "MMD-Polynomial":  lambda x, y: mmd(x, y, kernel=poly),
    "TV (KDE-MC approx)": tv_distance,
}
```

In [50]:
```
def plot_metric_panel(grids, std_grid, radius_grid, mode='imshow', cmap_name
    n_metrics = len(grids)
    ncols = 3
    nrows = int(np.ceil(n_metrics / ncols))
    fig, axes = plt.subplots(
        nrows, ncols, figsize=(5.8 * ncols, 4.7 * nrows), constrained_layout
    )
    axes = np.atleast_1d(axes).ravel()

    extent = [radius_grid.min(), radius_grid.max(), std_grid.min(), std_gric
    X, Y = np.meshgrid(radius_grid, std_grid)

    for ax, (metric_name, grid_values) in zip(axes, grids.items()):
        if mode == 'contourf':
            im = ax.contourf(X, Y, grid_values, levels=20, cmap=cmap_name)
        else:
            im = ax.imshow(
                grid_values,
                origin='lower',
                extent=extent,
```

```
                aspect='auto',
                cmap=cmap_name,
            )
        ax.set_title(metric_name)
        ax.set_xlabel('vertex_radius')
        ax.set_ylabel('gaussian_std')
        fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)

    for ax in axes[n_metrics:]:
        ax.axis('off')

    plt.suptitle('Distance heatmaps over (gaussian_std, vertex_radius)', y=1
    plt.show()
```

In [51]:
```
metric_grids = compute_metric_grids(
    metrics=metric_catalog,
    std_grid=std_vals,
    radius_grid=radius_vals,
 )
plot_metric_panel(metric_grids, std_vals, radius_vals)
```



Exercise 8 — Discussion

All metrics agree on the broad trend: distance increases as `vertex_radius` grows (the triangle mixture spreads further from the origin) and decreases as `gaussian_std`

increases (the centered Gaussian broadens to cover more of the same support). The minimum-distance region sits near `vertex_radius ≈ 0, gaussian_std ≈ noise_scale` (≈ 0.25), where both clouds are essentially indistinguishable centered Gaussians.

Despite this shared trend, the metrics show clear qualitative differences:

- **W1 vs W2**: Both are sensitive to the full geometry of the distributions. W1 penalises displacement linearly and is therefore less sensitive to large separations than W2, which penalises quadratically. Their heatmaps have the same qualitative shape but W2 values grow faster with `vertex_radius`.
- **SW1 / SW2 vs W1 / W2**: The sliced distances produce qualitatively similar heatmaps to their exact Wasserstein counterparts, confirming that the random-projection approximation preserves sensitivity to shape. Because projections collapse the 2-D structure to 1-D, SW slightly underestimates the exact Wasserstein value but captures the same trends.
- **MMD-RBF / MMD-Laplacian**: With the default bandwidth σ = 1, these kernels are sensitive to distributional differences at scales comparable to σ. The heatmaps are smooth and begin to saturate when the distributions are far apart (kernel evaluations become near-zero). The Laplacian decays more slowly than the RBF so it remains sensitive over a wider range of `vertex_radius`. Both are characteristic kernels, so they do define a true distance.
- **MMD-Linear**: The linear kernel k(x,y) = x·y only measures differences in first moments (means). Since both distributions are centered near the origin for all parameter values, MMD-Linear is nearly zero everywhere and completely fails to detect the shape difference between the triangle mixture and the centered Gaussian — it is **not** a useful distance for this problem.
- **MMD-Polynomial**: By capturing higher-order moments the polynomial kernel correctly detects distributional differences and produces a heatmap qualitatively similar to RBF/Laplacian, though with different absolute scaling.
- **TV (KDE-MC approx)**: TV is bounded in [0, 1] and is the most discriminative — it detects any non-zero probability-mass difference. Its heatmap saturates quickly (reaches ≈ 1) for even moderate `vertex_radius`, making it hard to distinguish moderate from large separations. Wasserstein distances remain informative over a much wider dynamic range.

In summary: all characteristic-kernel MMD variants and the Wasserstein/SW distances are sensitive to the full shape of the distributions, but **MMD-Linear is blind to any distributional difference beyond the mean**. TV is highly discriminative but saturates

early and loses resolution at large separations. The Wasserstein distances provide the richest geometric information across the full parameter grid.

The Curse of Dimensionality

The empirical Wasserstein distance converges to the true continuous Wasserstein distance at a rate of roughly $n^{-1/d}$, where n is the number of samples and d is the dimension. This represents the *curse of dimensionality*, because for large dimensions, the convergence will be very slow.

To illustrate this, we will focus on the example of computing the distance between $\mu = \mathcal{N}(0, I_d)$ and **itself** (e.g. $W_p(\mu, \mu)$). If we had access to the full measure μ, the distance should be 0. However, since we have finitely many samples to *approximate this value*, we will observe how fast our *empirical* metrics approach 0 as the number of samples increases.

In [52]:
```python
# PLOTTING FUNCTIONS
def plot_dimension_convergence(sample_sizes, results_by_dim, dimensions, yla
    """Plot mean convergence curves with trial-based error bars on a log-log
    plt.figure()
    for d in dimensions:
        trial_matrix = np.asarray(results_by_dim[d], dtype=float)
        means = trial_matrix.mean(axis=1)
        stds = trial_matrix.std(axis=1, ddof=1) if trial_matrix.shape[1] > 1

        # Keep lower error bars positive for log-scale plotting.
        yerr = np.minimum(stds, 0.95 * means)
        label = f'{label_prefix}d={d}' if label_prefix else f'd={d}'
        line = plt.loglog(sample_sizes, means, marker=marker, linestyle=line
        plt.errorbar(
            sample_sizes,
            means,
            yerr=yerr,
            fmt='none',
            ecolor=line.get_color(),
            elinewidth=1.2,
            capsize=3,
            alpha=0.8,
        )

    plt.xlabel('Number of samples (n)')
    plt.xlim(left=max(sample_sizes[0]-10, 1))
    plt.ylabel(ylabel)
    plt.title(title)
    plt.legend()
    plt.show()
```

In [53]:
```python
# PLOTTING FUNCTIONS
def plot_scaling_comparison(n_vals, d_max, series, title=None):
    """Plot empirical scaling curves and their fixed-rate fits."""
    plt.figure(figsize=(10, 6))
```

```
    for spec in series.values():
        data_label = spec['data_label'].replace('{d}', str(d_max))
        fit_label = spec['fit_label'].replace('{d}', str(d_max))

        line = plt.loglog(
            n_vals,
            spec['values'],
            f"{spec['marker']}-",
            label=data_label,
        )[0]
        plt.loglog(
            n_vals,
            spec['fit'],
            '--',
            color=line.get_color(),
            alpha=0.2,
            label=fit_label,
        )

    plt.xlabel('Number of samples (n)')
    plt.ylabel('Distance (log scale)')
    plt.title(title or f'Scaling with n at max dimension d={d_max}')
    plt.legend()
    plt.show()

def fit_power_law_with_fixed_exponent(n, y, alpha):
    """Fit y ~ C * n^{-alpha} by estimating C in log-space."""
    log_c = np.mean(np.log(y) + alpha * np.log(n))
    c = np.exp(log_c)
    return c * n ** (-alpha)


def plot_scaling_from_results(results, rates, d_max):
    """Build and plot scaling comparison from experiment outputs."""
    n_vals = np.asarray(results['sample_sizes'], dtype=float)

    labels = {
        'W2': {
            'marker': 'o',
            'data_label': 'Empirical W2 (d={d})',
            'fit_label': r'Fit $\propto n^{-1/{d}}$',
        },
        'SW2': {
            'marker': 's',
            'data_label': 'Empirical SW2 (d={d})',
            'fit_label': r'Fit $\propto n^{-1/2}$ (SW2)',
        },
        'MMD': {
            'marker': '^',
            'data_label': 'Empirical MMD (d={d})',
            'fit_label': r'Fit $\propto n^{-1/2}$ (MMD)',
        },
    }

    series = {}
```

```
    for metric_name, style in labels.items():
        metric_mean = np.asarray(results[metric_name][d_max], dtype=float).m
        series[metric_name] = {
            'values': metric_mean,
            'fit': fit_power_law_with_fixed_exponent(n_vals, metric_mean, ra
            **style,
        }
    plot_scaling_comparison(n_vals, d_max, series)
```

Exercise 9. Complete the function `run_dimension_experiment`, to do the following:

- Iterate over `dimensions` and `sample_sizes` and also over `range(trials)`.
- At each trial for a given value of `d`, `n`, sample `x` and `y` as standard gaussians $\mathcal{N}(0,1)$; and compute the `metric_fn(x,y)`.
- You should save the results in a dictionary: `results_by_dim` that maps each dimension `d` to an array of shape `(len(sample_sizes), trials)`.

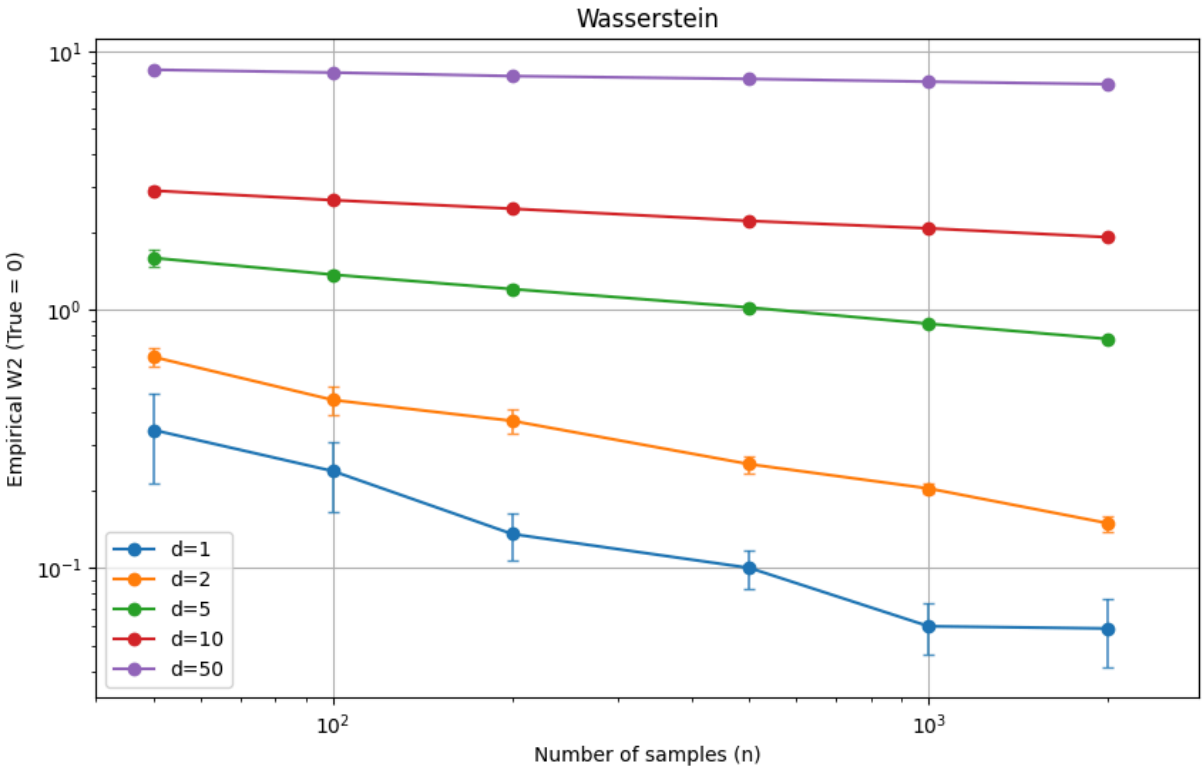
In [54]:
```
def run_dimension_experiment(dimensions, sample_sizes, trials, metric_fn):
    """Run a dimension-varying experiment and return per-trial metric values
    dimensions: List of dimensions to test.
    sample_sizes: List of sample sizes to test.
    trials: Number of independent trials to run for each (dimension, sample_
    metric_fn: Function that takes two point clouds and returns the distance

    Returns a dict mapping each dimension d to an array of shape (len(sample
        containing the distance metric values for each trial and sample size
    """
    results_by_dim = {
        d: np.empty((len(sample_sizes), trials), dtype=float) for d in dimer
    }

    for d in dimensions:
        for i, n in enumerate(sample_sizes):
            for t in range(trials):
                x = np.random.randn(n, d)
                y = np.random.randn(n, d)
                results_by_dim[d][i, t] = metric_fn(x, y)

    return results_by_dim
```

In [55]:
```
## Let's define the parameters for all the runs!
dimensions = [1, 2, 5, 10, 50]
sample_sizes = [50, 100, 200, 500, 1000, 2000]
trials = 5
```

Exercise 10. a) Run the experiment for the Wasserstein distance. Comment on what you observe.

In [56]:
```
results_w2 = run_dimension_experiment(
    dimensions=dimensions,
    sample_sizes=sample_sizes,
    trials=trials,
    metric_fn=wasserstein_p_empirical
```

```
)

plot_dimension_convergence(
    sample_sizes=sample_sizes,
    results_by_dim=results_w2,
    dimensions=dimensions,
    ylabel='Empirical W2 (True = 0)',
    title='Wasserstein',
)
```



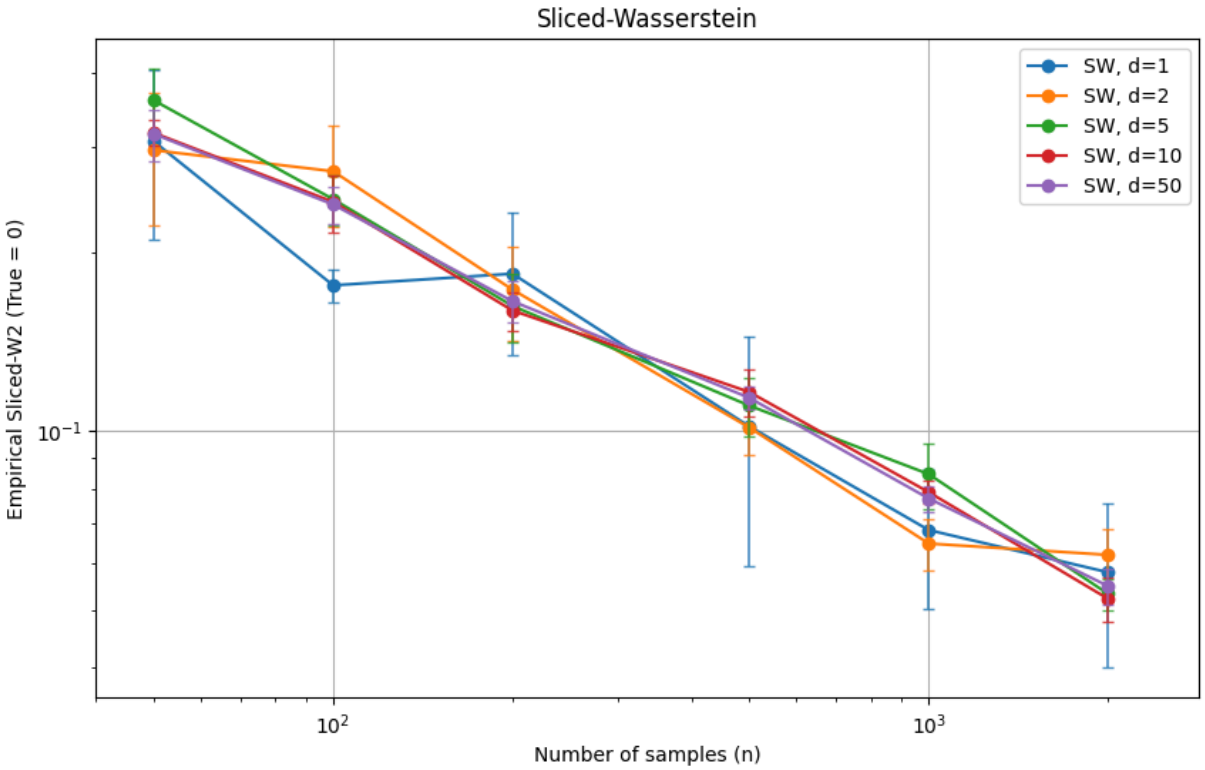
Exercise 10a — Wasserstein W2

All curves decrease as n grows (more samples reduce the finite-sample bias), but the rate slows dramatically with dimension. At $d = 1$ the empirical W2 collapses quickly, while at $d = 50$ it barely moves even at $n = 2000$. On the log-log scale the slopes are visibly different — roughly $-1/d$ per dimension — and the curves are well-separated, with high-d lines sitting far above low-d ones for the same n. This is the curse of dimensionality: to achieve the same approximation quality you need exponentially more samples as d grows.

Exercise 10. b) Run the experiment for the Sliced Wasserstein distance. Comment on what you observe.

In [57]:
```
# --- SOLUTION ---
results_sw = run_dimension_experiment(
    dimensions=dimensions,
    sample_sizes=sample_sizes,
    trials=trials,
    metric_fn=lambda x, y: sliced_wasserstein_p(x, y, n_projections=50)
```

```
)

plot_dimension_convergence(
    sample_sizes=sample_sizes,
    results_by_dim=results_sw,
    dimensions=dimensions,
    ylabel='Empirical Sliced-W2 (True = 0)',
    title='Sliced-Wasserstein',
    label_prefix='SW, ',
)
```



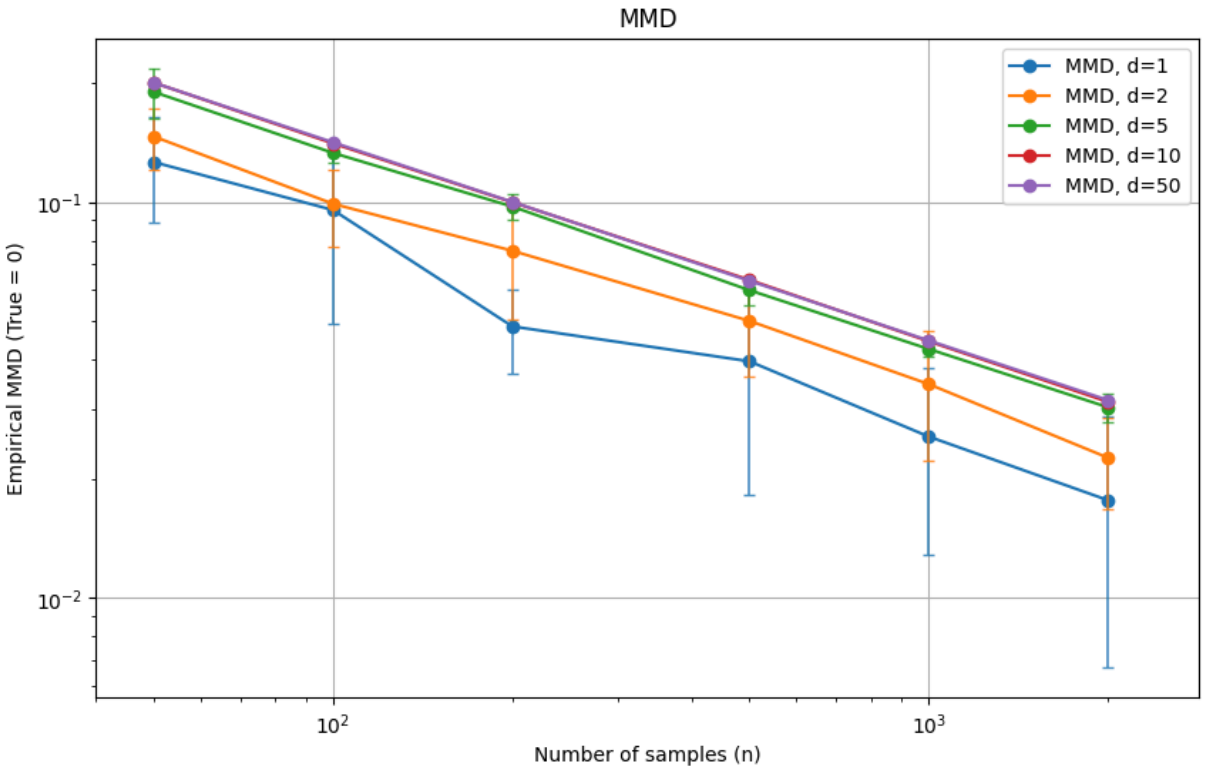
Exercise 10b — Sliced Wasserstein SW2

Unlike W2, the convergence curves for different dimensions cluster much closer together and share roughly the same log-log slope of $-1/2$, regardless of d. This reflects the theoretical $O(n^{-1/2})$ dimension-free convergence rate of SW: because the problem is always reduced to a collection of 1-D comparisons, the curse of dimensionality is sidestepped in terms of the convergence rate. The absolute values of SW2 do still grow slightly with d (the constant prefactor increases), but the rate at which the bias shrinks with n is the same across all dimensions.

Exercise 10. c) Run the experiment for the MMD. Comment on what you observe.

In [58]:
```
sigma=1.0
results_mmd = run_dimension_experiment(
    dimensions=dimensions,
    sample_sizes=sample_sizes,
    trials=trials,
    metric_fn=lambda x, y: mmd(x, y, kernel=lambda a, b: rbf(a, b, sigma=sig
```

```
)

plot_dimension_convergence(
    sample_sizes=sample_sizes,
    results_by_dim=results_mmd,
    dimensions=dimensions,
    ylabel='Empirical MMD (True = 0)',
    title='MMD',
    label_prefix='MMD, ',
)
```



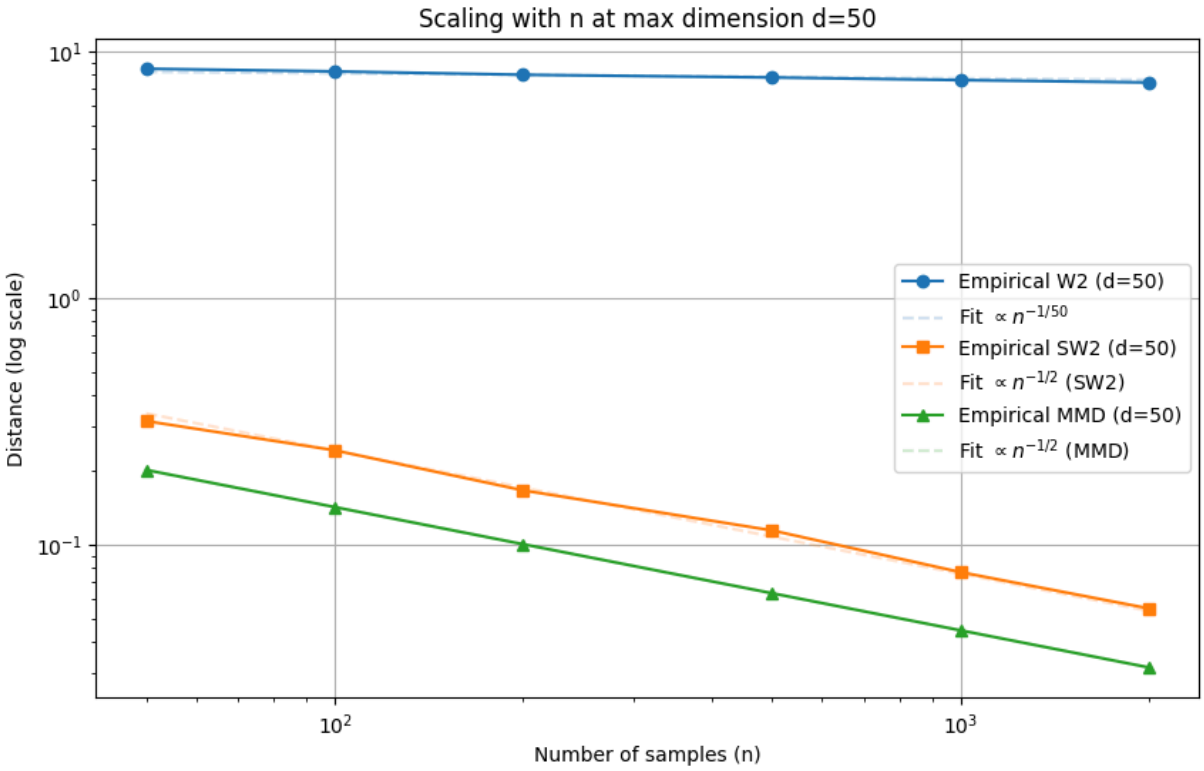
Exercise 10c — MMD (RBF kernel, $\sigma = 1$)

Like SW, all curves have a log-log slope close to $-1/2$, confirming the $O(n^{-1/2})$ dimension-free convergence rate of the empirical MMD. However, the absolute MMD values decrease noticeably as d grows. This is a consequence of the RBF kernel: for standard Gaussians, $\|x - y\|^2 \approx 2d$, so $k(x, y) = \exp(-\|x - y\|^2/2) \approx e^{-d} \to 0$ as d increases. The kernel itself concentrates near zero in high dimensions, shrinking the range of MMD values and reducing sensitivity — a subtler form of dimensionality dependence hidden in the constant rather than the rate.

Exercise 11. Run the following cell to observe the scaling of the distance values for a given value of `d`. Comment on the observed scalings. Did these *alternative* measures (SW and MMD) allow us to *escape* the curse of dimensionality?

In [59]:
```
scaling_results = {
    'sample_sizes': sample_sizes,
    'W2': results_w2,
    'SW2': results_sw,
```

```
    'MMD': results_mmd,
}
d_val = max(dimensions)

rates = {
    'W2': 1.0 / d_val,
    'SW2': 0.5,
    'MMD': 0.5,
}

plot_scaling_from_results(scaling_results, rates, d_max=d_val)
```



Exercise 11 — Scaling comparison at $d = 50$

At the largest dimension, the three metrics tell very different stories. W2 has a slope close to $-1/50$, nearly flat — matching the theoretical $n^{-1/d}$ rate and confirming severe curse of dimensionality: the empirical W2 barely improves even with thousands of samples. SW2 and MMD both have slopes close to $-1/2$, matching their theoretical dimension-free rate and tracking their respective reference fits well.

Did SW and MMD escape the curse? Partially. Both escape the curse *in convergence rate* — their empirical bias shrinks at $O(n^{-1/2})$ regardless of d, while W2 degrades to $O(n^{-1/50})$ at $d = 50$. However, neither escapes it entirely: SW's constant prefactor grows with d, and MMD's kernel values shrink exponentially with d (for fixed σ), reducing its discriminative power. So SW and MMD are far more sample-efficient than W2 in high dimensions, but they trade the curse of dimensionality in the convergence rate for a milder dependence on d in the constant.